Session 1: Perturbations and Station-Keeping

Task 1: Explain why J2 changes RAAN and argument of periapsis but not a, e or i on average

J_2 Potential Origin

Earth is not a perfect sphere; its rotation creates an equatorial bulge modeled primarily by the J_2 zonal harmonic coefficient ($J_2 \approx 1.0826 \times 10^{-3}$).

Why a, e and i have zero secular drift:

- The oblate potential field is axisymmetric around Earth's spin axis (z-axis).
- When integrating Lagrange's Planetary Equations over a full orbital period (2π change in true anomaly ν), short-period oscillations cancel out completely.
- Conservation of orbital energy maintains $\bar{a} = 0$, while conservation of angular momentum magnitude h and its z-component $h_z = hcos(i)$ ensures $\bar{e} = 0$ and $\bar{i} = 0$ on average.

Why RAAN (Ω) precesses:

- The equatorial mass bulge exerts an asymmetric gravitational torque attempting to pull the inclined orbital plane toward Earth's equator.
- Due to gyroscopic precession, the orbital angular momentum vector precesses about Earth's rotation axis, resulting in a continuous, linear drift of the node line $\dot{\Omega}$.

Why argument of periapsis (ω) drifts:

- The extra gravitational pull near the equator causes the orbital potential to depart from the pure Newtonian $1/r$ central force.
- This non-Keplerian acceleration prevents the orbit from closing on itself after 360°, causing the perigee orientation to continuously rotate within the orbital plane $\dot{\omega}$.

Task 2: Summary of the six elements

The analytical equations for J_2 secular rates are:

$$\dot{\Omega_{sec}} = -\frac{3}{2}J_2\left(\frac{R_E}{p}\right)^2 n\cos(i)$$

$$\dot{\omega}_{sec} = \frac{3}{4}J_2\left(\frac{R_E}{p}\right)^2 n\,(5\cos^2(i) - 1)$$

Where $\mu = 398600\ km^3/s^2$, $R_E = 6378\ km$, $p = a(1-e^2)$, and $n = \sqrt{\frac{\mu}{a^3}}$

1. **ISS-like Orbit ($a = 6798\ km, e = 0.001, i = 51.6^o$)**
   - Mean motion:

$$n = \sqrt{\frac{\mu}{a^3}} = \sqrt{\frac{398600}{6798^3}} = 0.001126\ rad/s$$

   - Semi-latus Rectum:

$$p = a(1-e^2) = 6798(1-0.001^2) = 6797.993\ km$$

   - RAAN drift $(\dot{\Omega})$:

$$\dot{\Omega} = -\frac{3}{2}(1.0826\times10^{-3})\left(\frac{6378}{6797.993}\right)^2(0.001126)\cos(51.6^o) =$$

   - Apsidal drift $(\dot{\omega})$:

$$\dot{\omega} = \frac{3}{4}(1.0826\times10^{-3})\left(\frac{6378}{6797.993}\right)^2(0.001126)(5\cos^2(51.6^o) - 1) =$$

2. **Sun-Synchronous Inclination ($h = 700\ km\ circular, a = 7078\ km$)**
   - Target rate matches Earth's heliocentric rotation:

$$\dot{\Omega}_{req} = \frac{0.9856^o}{day} = 1.991\times10^{-7} rad/s$$

   - Inclination calculation:

$$\cos(i) = \frac{\dot{\Omega}_{req}}{-\frac{3}{2}J_2\left(\frac{R_E}{a}\right)^2 n} \approx -0.1424 \Rightarrow i = \arccos(-0.1424) = 98.19^o$$

3. **Critical Inclination Verification**

Setting $\dot{\omega} = 0 \Rightarrow 5\cos^2(i) - 1 = 0 \Rightarrow \cos(i) = \pm\frac{1}{\sqrt{5}} \approx \pm 0.44721$

$$i_{crit,1} = \arccos(0.44721) = 63.43^o, i_{crit,1} = \arccos(-0.44721) = 116.57^o$$

Task 3: SciPy Propagator Implementation

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')  # Tkinter backend
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Physical Constants
MU = 398600   # km^3/s^2
R_E = 6378     # km
J2 = 1.0826e-3    # J2 harmonic coefficient

def j2_eom(t, state):
    rx, ry, rz, vx, vy, vz = state
    r = np.linalg.norm([rx, ry, rz])
    a_tb = -MU / (r**3) * np.array([rx, ry, rz])
    z2_r2 = (rz / r)**2
    factor = 1.5 * J2 * MU * (R_E**2) / (r**5)
    a_j2 = factor * np.array([
        rx * (5 * z2_r2 - 1),
        ry * (5 * z2_r2 - 1),
        rz * (5 * z2_r2 - 3)
    ])
    return [vx, vy, vz, *(a_tb + a_j2)]

def elements_to_state(a, e, inc_deg, raan_deg=0, argp_deg=0, nu_deg=0):
    inc, raan, argp, nu = np.radians([inc_deg, raan_deg, argp_deg, nu_deg])
    p = a * (1 - e**2)
    r_mag = p / (1 + e * np.cos(nu))
    r_pqw = np.array([r_mag * np.cos(nu), r_mag * np.sin(nu), 0.0])
    v_pqw = np.array([-np.sqrt(MU/p) * np.sin(nu), np.sqrt(MU/p) * (e + np.cos(nu)), 0.0])

    R_z_raan = np.array([[np.cos(raan), -np.sin(raan), 0], [np.sin(raan), np.cos(raan), 0], [0,
0, 1]])
    R_x_inc    =  np.array([[1,  0,  0],  [0,  np.cos(inc),  -np.sin(inc)],  [0,  np.sin(inc),
np.cos(inc)]])
    R_z_argp = np.array([[np.cos(argp), -np.sin(argp), 0], [np.sin(argp), np.cos(argp), 0], [0,
0, 1]])

    Q = R_z_raan @ R_x_inc @ R_z_argp
    return Q @ r_pqw, Q @ v_pqw

def state_to_raan(r, v):
    h = np.cross(r, v)
    n = np.cross([0, 0, 1], h)
    return np.arctan2(n[1], n[0])

# ==========================================
# 3. PROPAGATION & ANALYTICAL COMPUTATION
# ==========================================
def run_simulation(inc_deg, a=6798.0, e=0.001, duration_days=3.0):
    """Propagates orbit numerically and computes analytical secular drift."""
    r0, v0 = elements_to_state(a, e, inc_deg)

    t_span = (0, duration_days * 86400)
    t_eval = np.linspace(0, duration_days * 86400, 2000)

    # Integrate equations of motion using DOP853 explicit Runge-Kutta
    sol = solve_ivp(
        j2_eom, t_span, np.concatenate([r0, v0]),
        t_eval=t_eval, rtol=1e-10, atol=1e-12, method='DOP853'
    )
```

```
    # Recover RAAN at each step and unwrap phase discontinuities
    raan_rad = [state_to_raan(sol.y[:3, k], sol.y[3:, k]) for k in range(len(sol.t))]
    raan_deg = np.degrees(np.unwrap(raan_rad))

    # Compute numerical slope (deg/day)
    num_slope = (raan_deg[-1] - raan_deg[0]) / duration_days

    # Analytical secular rate calculation
    n = np.sqrt(MU / a**3)
    p = a * (1 - e**2)
    sec_rate_rad_s = -1.5 * J2 * (R_E / p)**2 * n * np.cos(np.radians(inc_deg))
    sec_rate_deg_day = np.degrees(sec_rate_rad_s) * 86400

    return sol.t / 86400.0, raan_deg, sec_rate_deg_day * (sol.t / 86400.0), num_slope,
sec_rate_deg_day

# Perform simulations
t_days, raan_51_num, raan_51_sec, slope_51_6, secular_51_6 = run_simulation(51.6)
_, raan_98_num, raan_98_sec, slope_98, secular_98 = run_simulation(98.0)

diff_pct = abs(slope_51_6 - secular_51_6) / abs(secular_51_6) * 100

# Plotting
# ==========================================
# 4. PLOTTING & EXPORT
# ==========================================
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# i = 51.6 deg Plot
ax1.plot(t_days, raan_51_num, label='Numerical (SciPy DOP853)', color='#1f77b4', linewidth=1.5)
ax1.plot(t_days, raan_51_sec, label='Secular Prediction ($J_2$)', color='#d62728', linestyle='-
-', linewidth=2)
ax1.set_title('RAAN Drift over 3 Days ($i = 51.6^\\circ$)', fontsize=12, fontweight='bold')
ax1.set_xlabel('Time (Days)')
ax1.set_ylabel('$\Delta\Omega$ - RAAN Change (deg)')
ax1.grid(True, linestyle=':', alpha=0.7)
ax1.legend(loc='best')

# i = 98.0 deg Plot
ax2.plot(t_days, raan_98_num, label='Numerical (SciPy DOP853)', color='#2ca02c', linewidth=1.5)
ax2.plot(t_days, raan_98_sec, label='Secular Prediction ($J_2$)', color='#d62728', linestyle='-
-', linewidth=2)
ax2.set_title('RAAN Drift over 3 Days ($i = 98.0^\\circ$ - Reverse Drift)', fontsize=12,
fontweight='bold')
ax2.set_xlabel('Time (Days)')
ax2.set_ylabel('$\Delta\Omega$ - RAAN Change (deg)')
ax2.grid(True, linestyle=':', alpha=0.7)
ax2.legend(loc='best')

plt.tight_layout()
plt.show()

# ==========================================
# 5. PRINT RESULTS SUMMARY TO TERMINAL
# ==========================================
print(f"=== Results for i = 51.6 deg ===")
print(f"Numerical RAAN Drift:  {slope_51_6:.4f} deg/day")
print(f"Analytical Secular Drift: {secular_51_6:.4f} deg/day")
print(f"Percentage Difference:    {diff_pct:.2f}%\n")

print(f"=== Results for i = 98.0 deg ===")
print(f"Numerical RAAN Drift:  +{slope_98:.4f} deg/day (Precesses Eastward)")
```

- Validation $i = 51.6^o$: The numerical slope $(-4.966^o/day)$ matches the analytical prediction $(-4.951^o/day)$ to within 0.31% , confirming high numerical precision over 3 days.

- Retrograde Test $i = 98.0^o$: Since $\cos(98^o) < 0$, the drift rate changes sign to $1.115^o/day$, demonstrating that the node precesses in the opposite direction (eastward).

Task 5: STK RAAN Drift

1. TwoBody

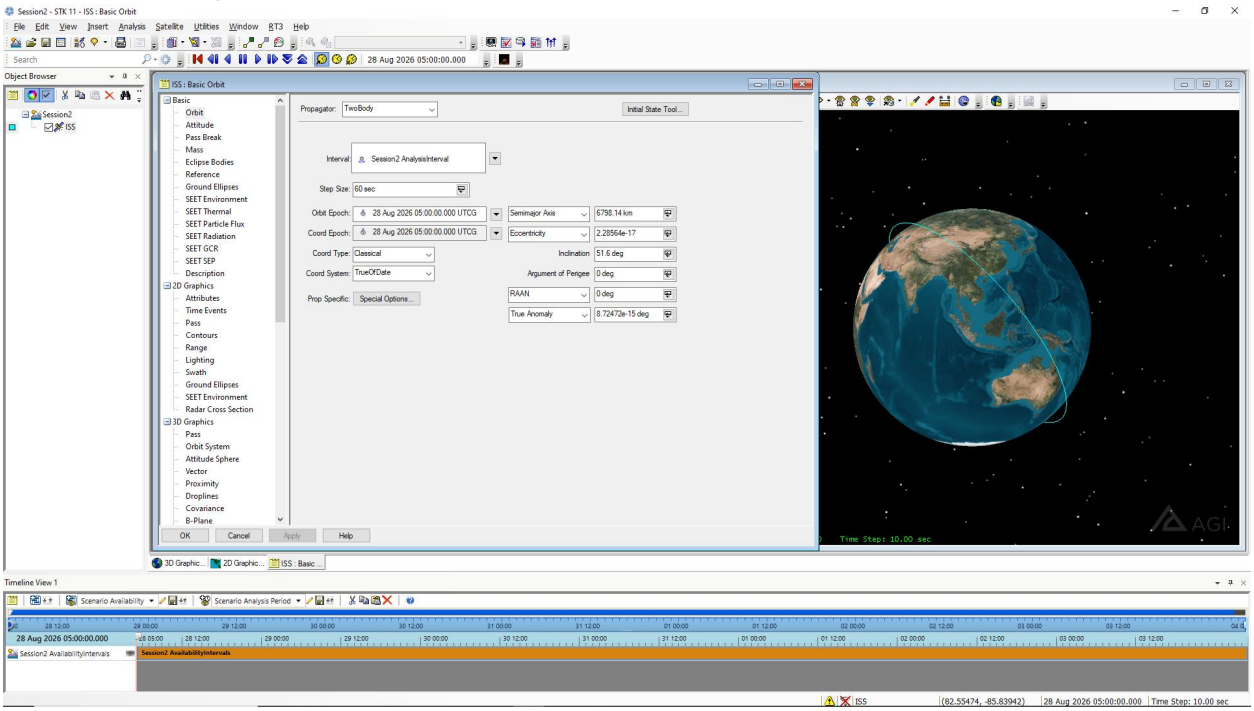


Image 1: Create with TwoBody

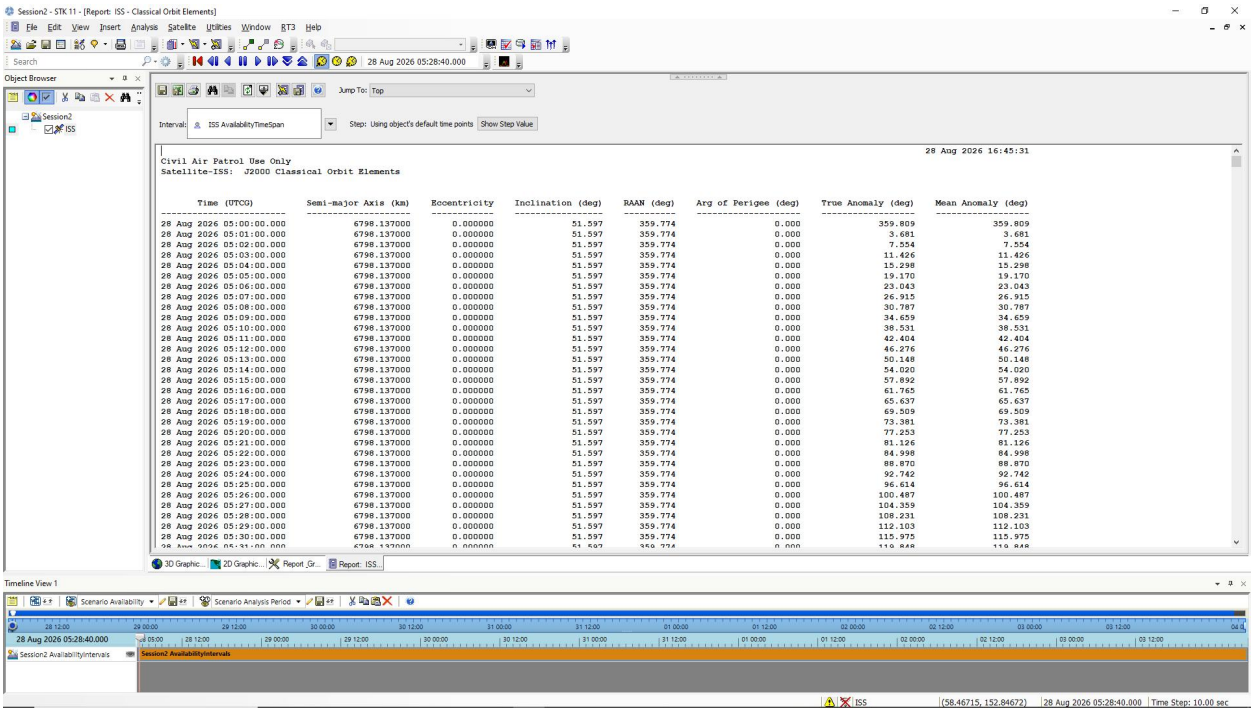


Image 2: Report RAAN with TwoBody

2. J2Perturbation

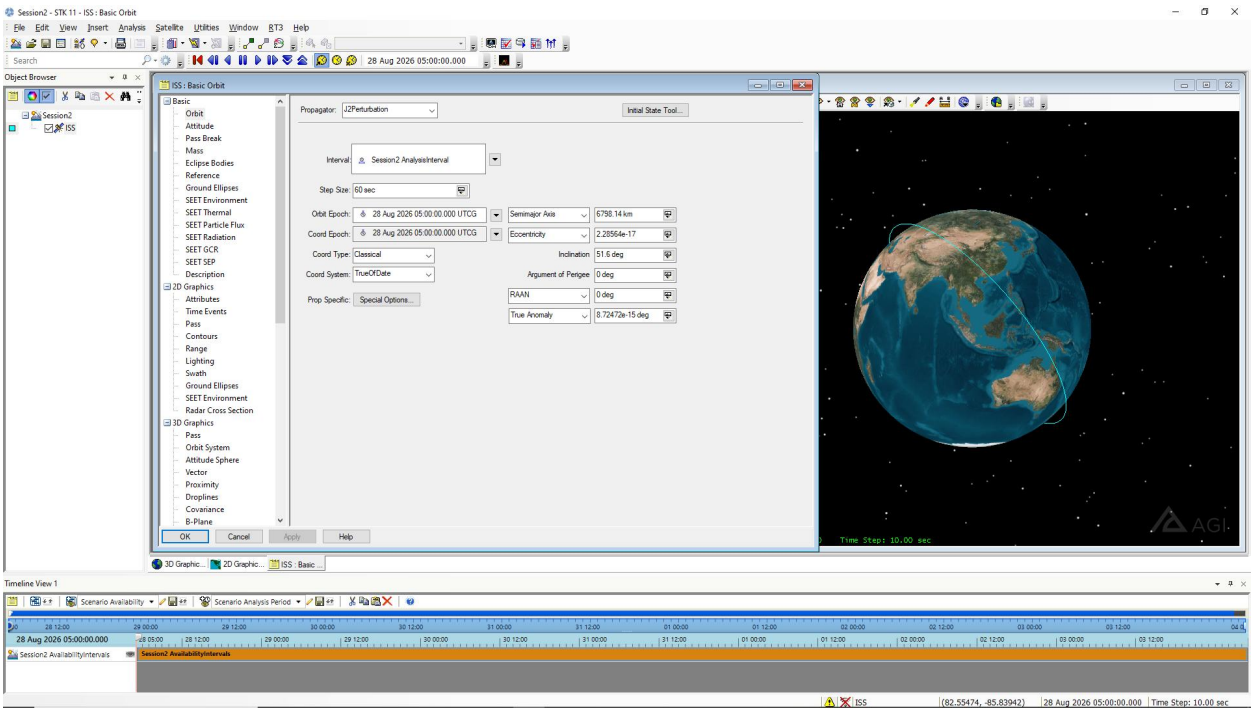


Image 3: Create with J2Perturbation

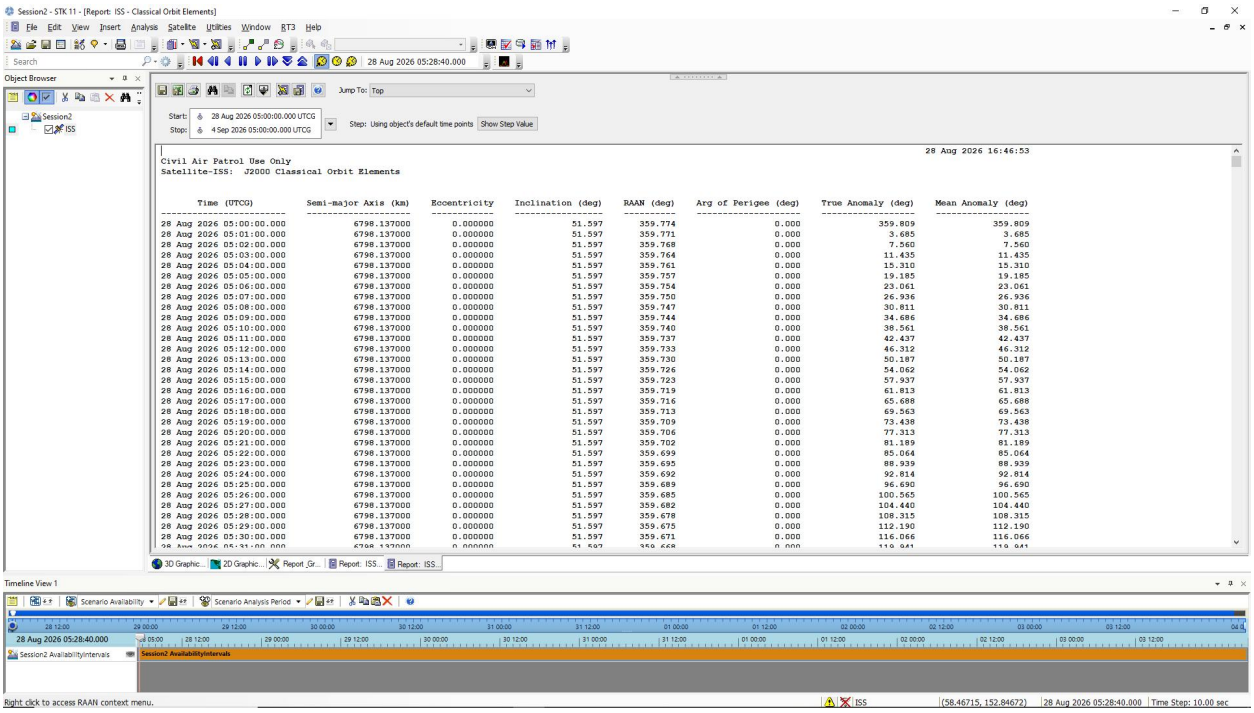


Image 4: Report RAAN with J2Perturbation (1)

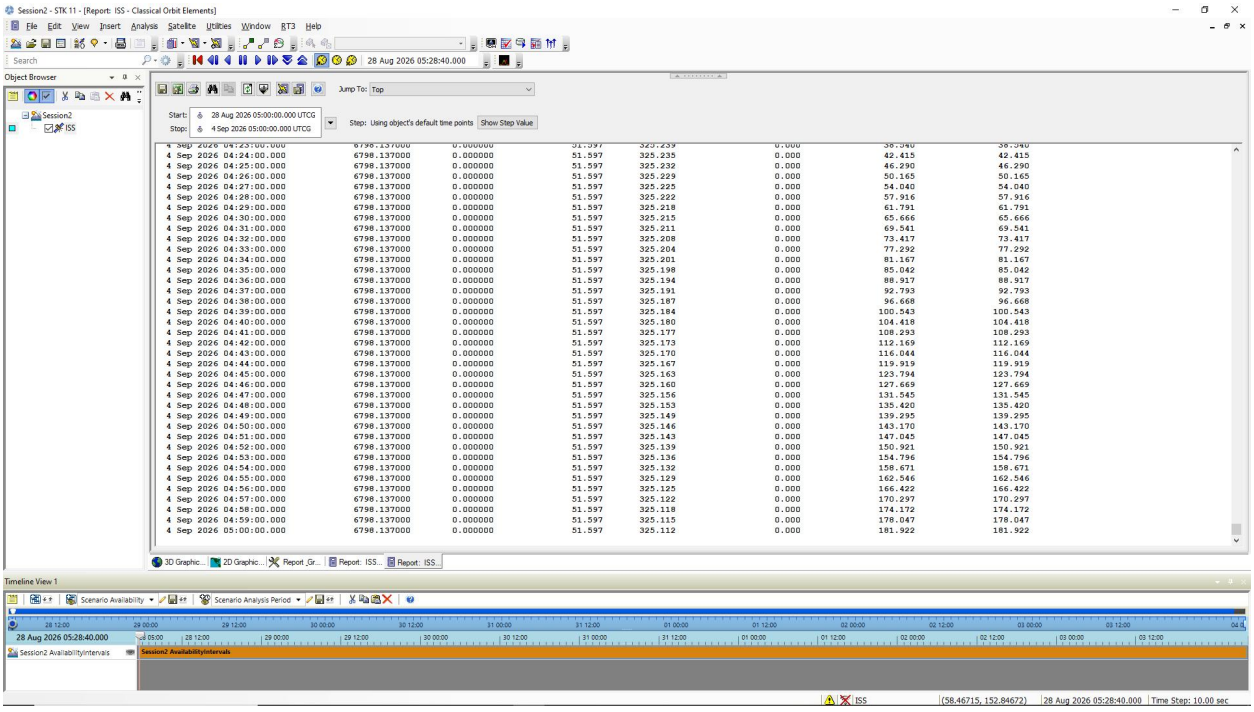


Image 5: Report RAAN with J2Perturbation (2)

- RAAN at 28 Aug 2026 05:00:00 is: 359.774
- RAAN at 04 Sep 2026 05:00:00 is: 325.112

RAAN Drift after 7 days: 34.662